MNEMO — ENHANCED MASTER BUILD DOCUMENT
"Your lectures, remembered forever."
This document extends the base Mnemo implementation plan with four groundbreaking additions:
1. **ATLAS Mode** — Conversational AI voice interface (Iron Man style)
2. **Spatial Voice Navigation** — Speak commands to fly through your 3D universe
3. **Collaborative Universes** — Real-time multiplayer study sessions
4. **AR Export** — View your knowledge graph overlaid on the real world

Read this before Antigravity executes anything. Base stack is unchanged. Add only what is here.

WHY THESE FEATURES WIN

Every other team will have a chat interface. Nobody will have a voice AI that talks back like Atlas, real-time 3D collaboration, or AR export. These are the three demo moments that make judges stop, turn around, and call their colleagues over. Each one is buildable in the time budget and costs ₹0.

The demo script with these features:
> "Watch. I'm going to upload this lecture... [3D universe appears] ...now I'm going to ask Atlas about the weakest node... [voice responds conversationally] ...now I'm going to study this with my friend in real time... [second cursor appears in the 3D space] ...and now I'm going to point my phone at my desk and walk through this knowledge graph in my room."

No team at a college hackathon has ever demoed that sequence. You will be the first.

ADDITION 1 — ATLAS MODE
Conversational AI Voice Interface

What it is: A persistent voice assistant embedded in the 3D universe. The user speaks naturally — "Atlas, which concept am I forgetting?" or "Explain photosynthesis like I'm five" or "Take me to the weakest node" — and Mnemo responds with a synthesised voice and takes action in the 3D space simultaneously.

The Atlas personality prompt (use this exactly):

```typescript
// lib/atlas.ts

const ATLAS_SYSTEM_PROMPT = `
You are ATLAS, the AI assistant embedded in Mnemo — a 3D spatial learning platform.
Your personality:
- Speak like a brilliant, warm friend who happens to know everything — not a textbook
- You address the user casually but with precision. Like Tony Stark's Atlas: efficient, witty, never condescending
```

```
- Short responses by default (2–3 sentences). Expand only when asked
- You always know the user's memory state and refer to it: "You've reviewed this twice but your retention is dropping — want to drill it?"
- Use light humour when appropriate: "That node has been sitting at 23% for three days. It's basically waving at you."
- Never say "I cannot" — reframe as "Here's what I can do instead"
- Refer to the 3D graph naturally: "flying over to...", "lighting up the...", "the cluster on your left..."

Available actions you can trigger (return these as JSON alongside your response):
- navigate_to_node: { nodeId: string }
- highlight_nodes: { nodeIds: string[], color: string }
- open_flashcard: { nodeId: string }
- zoom_to_cluster: { nodeIds: string[] }
- start_quiz: { nodeIds: string[] }
- speak_explanation: { text: string }

Always return JSON in this format alongside your spoken response:
{
 "speech": "your conversational response here",
 "action": { "type": "action_name", "payload": { ... } } | null,
 "emotion": "confident" | "encouraging" | "concerned" | "playful"
}

Current knowledge state: {KNOWLEDGE_STATE}
Current document: {DOCUMENT_TITLE}
`;

export interface AtlasResponse {
 speech: string;
 action: {
  type: string;
  payload: Record<string, unknown>;
 } | null;
 emotion: "confident" | "encouraging" | "concerned" | "playful";
}

export async function askAtlas(
 userSpeech: string,
 knowledgeState: Record<string, number>,
 documentTitle: string
): Promise<AtlasResponse> {
 const systemPrompt = ATLAS_SYSTEM_PROMPT
  .replace("{KNOWLEDGE_STATE}", JSON.stringify(knowledgeState))
  .replace("{DOCUMENT_TITLE}", documentTitle);

 const res = await fetch(
```

```
  `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?
key=${process.env.GEMINI_API_KEY}`,
  {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
   system_instruction: { parts: [{ text: systemPrompt }] },
   contents: [{ parts: [{ text: userSpeech }] }],
   generationConfig: { temperature: 0.7, maxOutputTokens: 512 },
  }),
  }
 );

 const data = await res.json();
 const raw = data.candidates?.[0]?.content?.parts?.[0]?.text ?? "{}";
 const cleaned = raw.replace(/```json|```/g, "").trim();

 try {
  return JSON.parse(cleaned);
 } catch {
  return {
   speech: "I'm having a moment. Try again.",
   action: null,
   emotion: "playful",
  };
 }
}
```

Voice input (Web Speech API — free, no backend needed):

```typescript
// hooks/useAtlasVoice.ts
"use client";

import { useState, useCallback, useRef } from "react";

export function useAtlasVoice(onTranscript: (text: string) => void) {
 const [listening, setListening] = useState(false);
 const recognitionRef = useRef<SpeechRecognition | null>(null);

 const startListening = useCallback(() => {
  const SpeechRecognition =
   window.SpeechRecognition || window.webkitSpeechRecognition;
  if (!SpeechRecognition) return;
```

```
  const recognition = new SpeechRecognition();
  recognition.continuous = false;
  recognition.interimResults = false;
  recognition.lang = "en-US";

  recognition.onresult = (e) => {
   const transcript = e.results[0][0].transcript;
   onTranscript(transcript);
  };

  recognition.onend = () => setListening(false);
  recognition.onerror = () => setListening(false);

  recognitionRef.current = recognition;
  recognition.start();
  setListening(true);
  }, [onTranscript]);

  const stopListening = useCallback(() => {
  recognitionRef.current?.stop();
  setListening(false);
  }, []);

  return { listening, startListening, stopListening };
}
```

Voice output (Web Speech Synthesis — free, no backend needed):

```typescript
// lib/speech-synthesis.ts

const ATLAS_VOICE_CONFIG = {
  rate: 0.95,  // Slightly slower than default — deliberate, authoritative
  pitch: 0.85,  // Slightly lower — gravitas without being robotic
  volume: 0.9,
};

export function speakAtlas(text: string, emotion: string): void {
  if (typeof window === "undefined") return;
  window.speechSynthesis.cancel(); // Stop any current speech

  const utterance = new SpeechSynthesisUtterance(text);
```

```
  // Find a British/neutral voice if available (Atlas feel)
  const voices = window.speechSynthesis.getVoices();
  const preferred = voices.find(
   (v) =>
   v.name.includes("Daniel") ||
   v.name.includes("Google UK") ||
   v.name.includes("Microsoft David")
  );
  if (preferred) utterance.voice = preferred;

  utterance.rate = ATLAS_VOICE_CONFIG.rate;
  utterance.pitch =
   emotion === "playful"
    ? ATLAS_VOICE_CONFIG.pitch + 0.1
    : ATLAS_VOICE_CONFIG.pitch;
  utterance.volume = ATLAS_VOICE_CONFIG.volume;

  window.speechSynthesis.speak(utterance);
}
```

Atlas UI Component — the floating orb:

```typescript
// components/ui/AtlasOrb.tsx
"use client";

import { motion, AnimatePresence } from "framer-motion";
import { useState, useCallback } from "react";
import { Mic, MicOff } from "lucide-react";
import { useAtlasVoice } from "@/hooks/useAtlasVoice";
import { askAtlas, AtlasResponse } from "@/lib/atlas";
import { speakAtlas } from "@/lib/speech-synthesis";

interface Props {
  knowledgeState: Record<string, number>;
  documentTitle: string;
  onAction: (action: AtlasResponse["action"]) => void;
}

const EMOTION_COLORS = {
  confident: "#6C63FF",
  encouraging: "#6BCB77",
  concerned: "#FFD93D",
  playful: "#00D4FF",
```

```
};

export function AtlasOrb({ knowledgeState, documentTitle, onAction }: Props) {
 const [thinking, setThinking] = useState(false);
 const [lastSpeech, setLastSpeech] = useState("Say 'Hey Atlas' to begin.");
 const [emotion, setEmotion] = useState<AtlasResponse["emotion"]>("confident");
 const [transcript, setTranscript] = useState("");

 const handleTranscript = useCallback(
  async (text: string) => {
  setTranscript(text);
  setThinking(true);

  const response = await askAtlas(text, knowledgeState, documentTitle);

  setLastSpeech(response.speech);
  setEmotion(response.emotion);
  setThinking(false);

  speakAtlas(response.speech, response.emotion);

  if (response.action) {
   onAction(response.action);
  }
  },
  [knowledgeState, documentTitle, onAction]
 );

 const { listening, startListening, stopListening } =
  useAtlasVoice(handleTranscript);

 const orbColor = EMOTION_COLORS[emotion];

 return (
  <div className="fixed bottom-6 right-6 z-30 flex flex-col items-end gap-3">
  {/* Speech bubble */}
  <AnimatePresence>
   {lastSpeech && (
   <motion.div
    initial={{ opacity: 0, y: 10, scale: 0.95 }}
    animate={{ opacity: 1, y: 0, scale: 1 }}
    exit={{ opacity: 0, scale: 0.95 }}
    className="max-w-xs p-3 bg-[#0A0A1F]/95 backdrop-blur border border-[#1E1E3F] rounded-2xl rounded-br-sm
text-sm text-[#F0F0FF]"
    style={{ borderColor: `${orbColor}40` }}
```

```
>
{thinking ? (
<span className="text-[#8888AA] italic">Processing...</span>
) : (
<>
{transcript && (
<p className="text-xs text-[#8888AA] mb-1 italic">
You: "{transcript}"
</p>
)}
<p>{lastSpeech}</p>
</>
)}
</motion.div>
)}
</AnimatePresence>

{/* The Orb */}
<motion.button
onClick={listening ? stopListening : startListening}
whileHover={{ scale: 1.08 }}
whileTap={{ scale: 0.95 }}
className="relative w-16 h-16 rounded-full flex items-center justify-center"
style={{
background: `radial-gradient(circle at 35% 35%, ${orbColor}60, ${orbColor}20)`,
boxShadow: listening
? `0 0 0 8px ${orbColor}20, 0 0 40px ${orbColor}60`
: `0 0 20px ${orbColor}40`,
border: `1px solid ${orbColor}60`,
}}
>
{/* Pulse ring when listening */}
{listening && (
<motion.span
className="absolute inset-0 rounded-full"
style={{ border: `2px solid ${orbColor}` }}
animate={{ scale: [1, 1.4], opacity: [0.8, 0] }}
transition={{ duration: 1, repeat: Infinity }}
/>
)}

{/* Thinking animation */}
{thinking ? (
<motion.div className="flex gap-1">
{[0, 1, 2].map((i) => (
```

```
    <motion.span
     key={i}
     className="w-1.5 h-1.5 rounded-full"
     style={{ backgroundColor: orbColor }}
     animate={{ y: [0, -6, 0] }}
     transition={{ duration: 0.6, delay: i * 0.15, repeat: Infinity }}
    />
    ))}
   </motion.div>
   ) : listening ? (
   <MicOff size={22} className="text-white" />
   ) : (
   <Mic size={22} className="text-white" />
   )}

   {/* ATLAS label */}
   <span
   className="absolute -top-6 left-1/2 -translate-x-1/2 text-[10px] font-mono tracking-widest"
   style={{ color: orbColor }}
   >
   ATLAS
   </span>
  </motion.button>
 </div>
 );
}
```

Wiring Atlas actions into the 3D universe (add to UniversePage):

```typescript
// Inside /app/universe/[documentId]/page.tsx — add this handler

const handleAtlasAction = useCallback(
 (action: AtlasResponse["action"]) => {
 if (!action || !graphData) return;

 switch (action.type) {
  case "navigate_to_node": {
   const node = graphData.nodes.find(
    (n) => n.id === action.payload.nodeId
   );
   if (node) setSelectedNode(node);
   break;
  }
```

```
  case "highlight_nodes": {
  // Pass highlighted IDs to KnowledgeUniverse for emissive boost
  setHighlightedNodes(action.payload.nodeIds as string[]);
  break;
  }
  case "open_flashcard": {
  router.push(`/flashcards/${documentId}?nodeId=${action.payload.nodeId}`);
  break;
  }
  case "zoom_to_cluster": {
  // Camera animation handled in KnowledgeUniverse via ref
  cameraRef.current?.zoomToNodes(action.payload.nodeIds as string[]);
  break;
  }
  }
 },
 [graphData, documentId, router]
);
```

Example Atlas conversations to demo to judges:

```
User: "Atlas, what should I study first?"
Atlas: "Mitochondria has been sitting at 17% for two days.
  It's basically haunting your graph. Want me to pull up
  a flashcard?" [navigates camera to that node, highlights it red]

User: "Explain the weakest concept simply"
Atlas: "Sure. Think of ATP synthesis like a spinning turbine
  at a dam — water pressure is the proton gradient,
  the turbine is ATP synthase, and electricity is your ATP."
  [highlights ATP synthesis node, pulses it gently]

User: "How am I doing overall?"
Atlas: "Honestly? Six out of twelve nodes are above 70%.
  The bottom four are pulling your mastery score down.
  Want to blitz through them now? Should take about 8 minutes."
```

ADDITION 2 — COLLABORATIVE UNIVERSES
Real-Time Multiplayer Study Sessions

What it is: Two or more users can enter the same 3D knowledge universe simultaneously. Each user is represented as a glowing cursor/avatar that other users can see moving in real time. When one user clicks a node, all users see it highlighted. A live chat panel floats over the 3D space.

Why it wins: Nobody else is building real-time 3D collaborative study. The judge demo moment: invite a second device on stage, show both avatars in the same universe simultaneously.

Implementation using Supabase Realtime (free tier — 500 concurrent connections):

```typescript
// lib/collaboration.ts

import { createClient } from "@supabase/supabase-js";

// Add to .env.local:
// NEXT_PUBLIC_SUPABASE_URL="https://xxx.supabase.co"
// NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJ..."
// Both are free from supabase.com

export const supabase = createClient(
 process.env.NEXT_PUBLIC_SUPABASE_URL!,
 process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

export interface CollaboratorState {
 userId: string;
 userName: string;
 avatarColor: string;
 selectedNodeId: string | null;
 cursorPosition: { x: number; y: number; z: number } | null;
 lastSeen: number;
}

// Presence colors — each collaborator gets a unique glow
export const COLLABORATOR_COLORS = [
 "#00D4FF", // cyan
 "#FF6B9D", // pink
 "#FFD93D", // yellow
 "#A78BFA", // violet
 "#6BCB77", // green
];

export function getCollabColor(index: number): string {
 return COLLABORATOR_COLORS[index % COLLABORATOR_COLORS.length];
}
```

```

**Collaboration hook:**

```typescript
// hooks/useCollaboration.ts
"use client";

import { useEffect, useState, useCallback, useRef } from "react";
import { supabase, CollaboratorState, getCollabColor } from "@/lib/collaboration";

export function useCollaboration(
 documentId: string,
 currentUser: { id: string; name: string }
) {
 const [collaborators, setCollaborators] = useState<
  Map<string, CollaboratorState>
 >(new Map());
 const channelRef = useRef<ReturnType<typeof supabase.channel> | null>(null);
 const myColor = useRef(
 getCollabColor(Math.floor(Math.random() * 5))
 );

 useEffect(() => {
 const channel = supabase.channel(`universe:${documentId}`, {
  config: { presence: { key: currentUser.id } },
 });

 channel
  .on("presence", { event: "sync" }, () => {
  const state = channel.presenceState<CollaboratorState>();
  const map = new Map<string, CollaboratorState>();
  Object.values(state)
   .flat()
   .forEach((user) => {
   if (user.userId !== currentUser.id) {
    map.set(user.userId, user);
   }
   });
  setCollaborators(map);
 })
 .subscribe(async (status) => {
 if (status === "SUBSCRIBED") {
  await channel.track({
   userId: currentUser.id,
```
```

```
    userName: currentUser.name,
    avatarColor: myColor.current,
    selectedNodeId: null,
    cursorPosition: null,
    lastSeen: Date.now(),
   } satisfies CollaboratorState);
  }
 });

 channelRef.current = channel;

 return () => {
  supabase.removeChannel(channel);
 };
}, [documentId, currentUser.id, currentUser.name]);

const broadcastNodeSelection = useCallback(
 async (nodeId: string | null) => {
  await channelRef.current?.track({
   userId: currentUser.id,
   userName: currentUser.name,
   avatarColor: myColor.current,
   selectedNodeId: nodeId,
   cursorPosition: null,
   lastSeen: Date.now(),
  });
 },
 [currentUser.id, currentUser.name]
);

 return {
  collaborators,
  myColor: myColor.current,
  broadcastNodeSelection,
 };
}
```

Collaborator cursors in the 3D scene (add to KnowledgeUniverse):

```typescript
// components/3d/CollaboratorCursor.tsx
"use client";

import { useRef } from "react";
```

```
import { Html } from "@react-three/drei";
import { useFrame } from "@react-three/fiber";
import * as THREE from "three";
import { CollaboratorState } from "@/lib/collaboration";

interface Props {
 collaborator: CollaboratorState;
 nodes: Array<{ id: string; positionX: number; positionY: number; positionZ: number }>;
}

export function CollaboratorCursor({ collaborator, nodes }: Props) {
 const meshRef = useRef<THREE.Mesh>(null);

 // Find the node this collaborator is looking at
 const targetNode = collaborator.selectedNodeId
  ? nodes.find((n) => n.id === collaborator.selectedNodeId)
  : null;

 const position: [number, number, number] = targetNode
  ? [targetNode.positionX, targetNode.positionY + 2, targetNode.positionZ]
  : [0, 0, 0];

 useFrame(() => {
 if (meshRef.current) {
  meshRef.current.rotation.y += 0.02;
 }
 });

 if (!targetNode) return null;

 return (
  <group position={position}>
  {/* Diamond cursor shape */}
  <mesh ref={meshRef}>
   <octahedronGeometry args={[0.3, 0]} />
   <meshStandardMaterial
    color={collaborator.avatarColor}
    emissive={collaborator.avatarColor}
    emissiveIntensity={1.2}
    transparent
    opacity={0.8}
   />
  </mesh>

  {/* Name label */}
```

```
  <Html distanceFactor={12} style={{ pointerEvents: "none" }}>
   <div
   className="px-2 py-0.5 rounded-full text-[10px] font-bold whitespace-nowrap"
   style={{
    backgroundColor: `${collaborator.avatarColor}20`,
    border: `1px solid ${collaborator.avatarColor}60`,
    color: collaborator.avatarColor,
    fontFamily: "Space Grotesk, sans-serif",
   }}
   >
   {collaborator.userName}
   </div>
  </Html>
 </group>
 );
}
```

Live chat panel (floats over the 3D space):

```typescript
// components/ui/CollabChat.tsx
"use client";

import { useState, useEffect, useRef } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { MessageCircle, Send, Users } from "lucide-react";
import { supabase } from "@/lib/collaboration";

interface Message {
 id: string;
 userId: string;
 userName: string;
 color: string;
 text: string;
 timestamp: number;
}

interface Props {
 documentId: string;
 currentUser: { id: string; name: string };
 myColor: string;
 collaboratorCount: number;
}
```

```
export function CollabChat({
 documentId,
 currentUser,
 myColor,
 collaboratorCount,
}: Props) {
 const [open, setOpen] = useState(false);
 const [messages, setMessages] = useState<Message[]>([]);
 const [input, setInput] = useState("");
 const bottomRef = useRef<HTMLDivElement>(null);

 useEffect(() => {
 const channel = supabase.channel(`chat:${documentId}`);

 channel
 .on("broadcast", { event: "message" }, ({ payload }) => {
 setMessages((prev) => [...prev.slice(-49), payload as Message]);
 })
 .subscribe();

 return () => { supabase.removeChannel(channel); };
 }, [documentId]);

 useEffect(() => {
 bottomRef.current?.scrollIntoView({ behavior: "smooth" });
 }, [messages]);

 const sendMessage = async () => {
 if (!input.trim()) return;
 const msg: Message = {
 id: crypto.randomUUID(),
 userId: currentUser.id,
 userName: currentUser.name,
 color: myColor,
 text: input.trim(),
 timestamp: Date.now(),
 };
 await supabase.channel(`chat:${documentId}`).send({
 type: "broadcast",
 event: "message",
 payload: msg,
 });
 setMessages((prev) => [...prev, msg]);
 setInput("");
 };
```

```
 return (
  <div className="fixed bottom-6 left-6 z-30">
  {/* Toggle button */}
  <motion.button
  onClick={() => setOpen((o) => !o)}
  whileHover={{ scale: 1.05 }}
  className="flex items-center gap-2 px-4 py-2.5 bg-[#0A0A1F]/90 backdrop-blur border border-[#1E1E3F]
rounded-xl text-sm text-[#8888AA] hover:text-[#F0F0FF] transition-all"
  >
  <Users size={16} className="text-[#00D4FF]" />
  <span>{collaboratorCount + 1} studying</span>
  <MessageCircle size={14} />
  </motion.button>

  {/* Chat panel */}
  <AnimatePresence>
  {open && (
  <motion.div
  initial={{ opacity: 0, y: 20, scale: 0.95 }}
  animate={{ opacity: 1, y: 0, scale: 1 }}
  exit={{ opacity: 0, y: 20, scale: 0.95 }}
  transition={{ type: "spring", stiffness: 300, damping: 30 }}
  className="absolute bottom-14 left-0 w-72 bg-[#0A0A1F]/95 backdrop-blur-xl border border-[#1E1E3F] rounded-
2xl overflow-hidden"
  >
  {/* Messages */}
  <div className="h-64 overflow-y-auto p-3 space-y-2">
  {messages.length === 0 && (
  <p className="text-xs text-[#8888AA] text-center pt-8">
  Study session started. Say something.
  </p>
  )}
  {messages.map((msg) => (
  <div key={msg.id} className="flex gap-2 text-xs">
  <span
  className="font-bold shrink-0"
  style={{ color: msg.color }}
  >
  {msg.userName.split(" ")[0]}:
  </span>
  <span className="text-[#F0F0FF]">{msg.text}</span>
  </div>
  ))}
  <div ref={bottomRef} />
```

```
    </div>

    {/* Input */}
    <div className="flex border-t border-[#1E1E3F]">
     <input
     value={input}
     onChange={(e) => setInput(e.target.value)}
     onKeyDown={(e) => e.key === "Enter" && sendMessage()}
     placeholder="Say something..."
     className="flex-1 px-3 py-2.5 text-xs bg-transparent text-[#F0F0FF] placeholder-[#8888AA] focus:outline-none"
     />
     <button
     onClick={sendMessage}
     className="px-3 text-[#6C63FF] hover:text-[#A78BFA] transition-all"
     >
     <Send size={14} />
     </button>
    </div>
   </motion.div>
   )}
  </AnimatePresence>
 </div>
 );
}
```

Session sharing — generate invite link:

```typescript
// In universe page — add share button to top bar
// Clicking this copies a URL like /universe/[documentId]?session=abc123
// Anyone with the link joins the same Supabase presence channel automatically

const handleShare = () => {
 const url = `${window.location.origin}/universe/${documentId}`;
 navigator.clipboard.writeText(url);
 // Show toast: "Link copied — anyone with this can study with you"
};
```

New .env.local entries for collaboration:
```bash
NEXT_PUBLIC_SUPABASE_URL="https://xxx.supabase.co"
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJ..."
# Both free from supabase.com — create project, copy from Settings > API
```

```

---

## ADDITION 3 — AR EXPORT
### View Your Knowledge Graph in Your Room

**What it is:** A button in the universe view that opens a WebXR/WebAR experience. The user's camera activates and the 3D knowledge graph is overlaid on whatever surface they point at — their desk, a wall, the floor. Nodes float in real space. Tap a node to expand it. Works on any modern phone without an app install.

**Why it wins:** No app download. No special hardware. Open a link on your phone, point at a surface, and your lecture becomes a 3D object in your room. Judges have never seen a student project do this.

**Implementation using model-viewer + WebXR (free, no APIs):**

```typescript
// app/ar/[documentId]/page.tsx
// This is a standalone page — no sidebar, full screen camera

"use client";

import { useEffect, useState } from "react";
import { useParams } from "next/navigation";
import Script from "next/script";

interface Node {
 id: string;
 label: string;
 explanation: string;
 importance: number;
 positionX: number;
 positionY: number;
 positionZ: number;
 memoryStrength?: number;
}

function getNodeColor(strength: number): string {
 if (strength < 0.3) return "#FF6B6B";
 if (strength < 0.6) return "#FFD93D";
 return "#6BCB77";
}

export default function ARPage() {
 const { documentId } = useParams();
```
```

```
const [nodes, setNodes] = useState<Node[]>([]);
const [supported, setSupported] = useState<boolean | null>(null);

useEffect(() => {
// Check WebXR support
setSupported("xr" in navigator);

fetch(`/api/graph/${documentId}/personalised`)
.then((r) => r.json())
.then(setNodes);
}, [documentId]);

// Scale positions to be AR-friendly (smaller than 3D universe)
const scale = 0.15;

return (
<>
<Script
type="module"
src="https://unpkg.com/@google/model-viewer/dist/model-viewer.min.js"
/>

<div className="fixed inset-0 bg-black flex flex-col">
{!supported ? (
<div className="flex-1 flex items-center justify-center p-8 text-center">
<div>
<p className="text-2xl font-bold text-[#F0F0FF] mb-2">
AR not supported
</p>
<p className="text-[#8888AA]">
Use Chrome on Android or Safari on iOS 15+ for AR mode.
</p>
</div>
</div>
) : (
<div className="flex-1 relative">
{/* AR Canvas using Three.js WebXR */}
<ARCanvas nodes={nodes} scale={scale} />

{/* Overlay instructions */}
<div className="absolute bottom-8 left-0 right-0 flex justify-center">
<div className="px-4 py-2 bg-black/60 backdrop-blur rounded-xl text-sm text-[#F0F0FF] text-center">
Point at a flat surface · Tap to place · Tap nodes to expand
</div>
</div>
```

```
  </div>
  )}
  </div>
  </>
 );
}
```

AR Canvas (WebXR with Three.js — no library cost):

```typescript
// components/3d/ARCanvas.tsx
"use client";

import { useEffect, useRef } from "react";
import * as THREE from "three";

interface Props {
 nodes: Array<{
  id: string;
  label: string;
  explanation: string;
  importance: number;
  positionX: number;
  positionY: number;
  positionZ: number;
  memoryStrength?: number;
 }>;
 scale: number;
}

export function ARCanvas({ nodes, scale }: Props) {
 const mountRef = useRef<HTMLDivElement>(null);

 useEffect(() => {
  if (!mountRef.current || nodes.length === 0) return;

  const scene = new THREE.Scene();
  const camera = new THREE.PerspectiveCamera(70, window.innerWidth / window.innerHeight, 0.01, 20);
  const renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });

  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.xr.enabled = true;
  mountRef.current.appendChild(renderer.domElement);
```

```
// Lighting
scene.add(new THREE.AmbientLight(0xffffff, 0.6));
const pointLight = new THREE.PointLight(0x6C63FF, 1);
pointLight.position.set(0, 2, 0);
scene.add(pointLight);

// Build graph group (placed on detected surface)
const graphGroup = new THREE.Group();

nodes.forEach((node) => {
 const strength = node.memoryStrength ?? 0.5;
 const color = strength < 0.3 ? 0xFF6B6B : strength < 0.6 ? 0xFFD93D : 0x6BCB77;
 const radius = ((node.importance / 10) * 0.04 + 0.02);

 const geometry = new THREE.SphereGeometry(radius, 16, 16);
 const material = new THREE.MeshStandardMaterial({
  color,
  emissive: color,
  emissiveIntensity: 0.4,
  roughness: 0.3,
  metalness: 0.4,
 });
 const sphere = new THREE.Mesh(geometry, material);
 sphere.position.set(
  node.positionX * scale,
  node.positionY * scale,
  node.positionZ * scale
 );

 graphGroup.add(sphere);
});

scene.add(graphGroup);

// XR session
const startAR = async () => {
 if (!("xr" in navigator)) return;
 const session = await (navigator as any).xr.requestSession("immersive-ar", {
  requiredFeatures: ["hit-test"],
  optionalFeatures: ["dom-overlay"],
  domOverlay: { root: document.body },
 });

 renderer.xr.setSession(session);
 renderer.setAnimationLoop(() => renderer.render(scene, camera));
```

```
  };

  startAR();

  return () => {
   renderer.dispose();
   mountRef.current?.removeChild(renderer.domElement);
  };
 }, [nodes, scale]);

 return <div ref={mountRef} className="w-full h-full" />;
}
```

AR launch button — add to universe top bar:

```typescript
// In UniversePage top bar — add alongside the Flashcards button

<a
 href={`/ar/${documentId}`}
 className="flex items-center gap-2 px-4 py-2 bg-[#00D4FF]/20 backdrop-blur border border-[#00D4FF]/60
rounded-xl text-sm text-[#00D4FF] hover:bg-[#00D4FF]/30 transition-all"
>
 {/* Use Glasses icon from lucide or a custom SVG */}
 <span>AR View</span>
</a>
```

Fallback for unsupported devices — QR code for phone:

```typescript
// If the user is on desktop, show a QR code to open AR on their phone
// Use qrcode library (free): npm install qrcode

import QRCode from "qrcode";

// Generate QR pointing to /ar/[documentId]
// Render as <img src={qrDataUrl} /> in a modal
// This is the demo move: "Scan this with your phone"
```

New dependency:
```bash
npm install qrcode @types/qrcode
```

```

---

## ADDITION 4 — ENHANCED VISUAL SYSTEM
### Pushing the Design to Porsche Level

The base design is strong. These additions make it genuinely memorable.

**4A. Particle field on node expansion:**

```typescript
// When a node is clicked, spawn particle burst at its position
// Add to ConceptNode.tsx

import { useRef, useState } from "react";
import { useFrame } from "@react-three/fiber";

// Particle burst component
function NodeBurst({ position, color, active }: {
 position: [number, number, number];
 color: string;
 active: boolean;
}) {
 const particles = useRef<THREE.Points>(null);
 const [opacity, setOpacity] = useState(1);

 const particleCount = 20;
 const positions = new Float32Array(particleCount * 3);
 const velocities = Array.from({ length: particleCount }, () => ({
 x: (Math.random() - 0.5) * 0.3,
 y: (Math.random() - 0.5) * 0.3,
 z: (Math.random() - 0.5) * 0.3,
 }));

 useFrame((_, delta) => {
 if (!active || !particles.current) return;
 const pos = particles.current.geometry.attributes.position;
 for (let i = 0; i < particleCount; i++) {
 pos.setX(i, (pos.getX(i) + velocities[i].x * delta));
 pos.setY(i, (pos.getY(i) + velocities[i].y * delta));
 pos.setZ(i, (pos.getZ(i) + velocities[i].z * delta));
 }
 pos.needsUpdate = true;
 setOpacity((o) => Math.max(0, o - delta * 1.5));
```
```

```
  });

  if (!active || opacity <= 0) return null;

  return (
   <points ref={particles} position={position}>
   <bufferGeometry>
    <bufferAttribute
     attach="attributes-position"
     args={[positions, 3]}
    />
   </bufferGeometry>
   <pointsMaterial
    color={color}
    size={0.05}
    transparent
    opacity={opacity}
   />
   </points>
  );
}
```

4B. Dynamic starfield that reacts to memory strength:

```typescript
// Replace static Stars with this in KnowledgeUniverse.tsx
// Stars pulse brighter when a node is selected, dimmer when in spatial recall mode

function DynamicStars({ intensity = 1 }: { intensity?: number }) {
  const starsRef = useRef<THREE.Points>(null);

  useFrame((_, delta) => {
   if (starsRef.current) {
    starsRef.current.rotation.y += delta * 0.005 * intensity;
   }
  });

  return (
   <Stars
    ref={starsRef}
    count={4000}
    depth={60}
    factor={intensity * 4}
    fade
```

```
  speed={0.3}
 />
 );
}
```

4C. Connection beam animation (replace static Line):

```typescript
// components/3d/ConceptEdge.tsx — animated flowing beam

"use client";

import { useRef } from "react";
import { useFrame } from "@react-three/fiber";
import { Line } from "@react-three/drei";
import * as THREE from "three";

interface Props {
 start: [number, number, number];
 end: [number, number, number];
 strength: number;
 highlighted?: boolean;
}

export function ConceptEdge({ start, end, strength, highlighted }: Props) {
 // Dashed flow effect — animate dashOffset over time
 const materialRef = useRef<any>(null);

 useFrame((_, delta) => {
  if (materialRef.current) {
   materialRef.current.dashOffset -= delta * 0.5;
  }
 });

 return (
  <Line
   points={[start, end]}
   color={highlighted ? "#A78BFA" : "#00D4FF"}
   lineWidth={highlighted ? strength * 2.5 : strength * 1.5}
   transparent
   opacity={highlighted ? 0.9 : 0.3 + strength * 0.3}
   dashed
   dashScale={2}
   dashSize={0.4}
```

```
 gapSize={0.2}
 />
 );
}
```

4D. Loading screen — the universe assembling itself:

```typescript
// components/ui/UniverseLoader.tsx
// Shown while graph data loads — nodes appear one by one

"use client";

import { motion } from "framer-motion";
import { useEffect, useState } from "react";

const MESSAGES = [
 "Mapping neural pathways...",
 "Calibrating memory vectors...",
 "Positioning concept nodes...",
 "Calculating relationship strength...",
 "Universe assembling...",
];

export function UniverseLoader() {
 const [msgIndex, setMsgIndex] = useState(0);

 useEffect(() => {
 const interval = setInterval(() => {
 setMsgIndex((i) => (i + 1) % MESSAGES.length);
 }, 1200);
 return () => clearInterval(interval);
 }, []);

 return (
 <div className="fixed inset-0 bg-[#050510] flex flex-col items-center justify-center z-50">
 {/* Animated constellation of dots */}
 <div className="relative w-32 h-32 mb-8">
 {Array.from({ length: 8 }).map((_, i) => (
 <motion.div
 key={i}
 className="absolute w-2 h-2 rounded-full bg-[#6C63FF]"
 style={{
 left: "50%",
```

```
    top: "50%",
    }}
    animate={{
    x: Math.cos((i / 8) * Math.PI * 2) * 50,
    y: Math.sin((i / 8) * Math.PI * 2) * 50,
    opacity: [0.3, 1, 0.3],
    scale: [0.8, 1.2, 0.8],
    }}
    transition={{
    duration: 2,
    delay: i * 0.15,
    repeat: Infinity,
    ease: "easeInOut",
    }}
   />
  ))}
  {/* Center pulse */}
  <motion.div
   className="absolute inset-0 flex items-center justify-center"
   animate={{ scale: [1, 1.1, 1] }}
   transition={{ duration: 2, repeat: Infinity }}
  >
   <div className="w-4 h-4 rounded-full bg-[#A78BFA]" />
  </motion.div>
 </div>

 <motion.p
  key={msgIndex}
  initial={{ opacity: 0, y: 5 }}
  animate={{ opacity: 1, y: 0 }}
  exit={{ opacity: 0 }}
  className="text-[#8888AA] text-sm font-mono"
 >
  {MESSAGES[msgIndex]}
 </motion.p>
</div>
 );
}
```

UPDATED SCHEMA ADDITIONS
Two new tables for collaboration

```typescript
// Add to /drizzle/schema.ts

// STUDY SESSIONS TABLE (collaborative)
export const studySessions = pgTable("study_sessions", {
 id: uuid("id").primaryKey().defaultRandom(),
 documentId: uuid("document_id")
  .references(() => documents.id, { onDelete: "cascade" })
  .notNull(),
 hostUserId: uuid("host_user_id")
  .references(() => users.id, { onDelete: "cascade" })
  .notNull(),
 sessionCode: text("session_code").notNull().unique(), // 6-char join code
 isActive: boolean("is_active").default(true),
 participantCount: integer("participant_count").default(1),
 createdAt: timestamp("created_at").defaultNow(),
 endedAt: timestamp("ended_at"),
});

// ATLAS CONVERSATION HISTORY TABLE (optional — for memory across sessions)
export const atlasHistory = pgTable("atlas_history", {
 id: uuid("id").primaryKey().defaultRandom(),
 userId: uuid("user_id")
  .references(() => users.id, { onDelete: "cascade" })
  .notNull(),
 documentId: uuid("document_id")
  .references(() => documents.id, { onDelete: "cascade" })
  .notNull(),
 userMessage: text("user_message").notNull(),
 atlasResponse: text("atlas_response").notNull(),
 actionTaken: text("action_taken"),
 createdAt: timestamp("created_at").defaultNow(),
});
```

UPDATED ENVIRONMENT VARIABLES

```bash
# .env.local — COMPLETE, all services free

# Neon Postgres
DATABASE_URL="postgresql://..."
```

```
# Gemini API (concepts + flashcards + Atlas)
GEMINI_API_KEY="AIza..."

# Better Auth
BETTER_AUTH_SECRET="your-32-char-random-secret"
BETTER_AUTH_URL="http://localhost:3000"
NEXT_PUBLIC_APP_URL="http://localhost:3000"

# Supabase (collaboration realtime) — from supabase.com free tier
NEXT_PUBLIC_SUPABASE_URL="https://xxx.supabase.co"
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJ..."
```

UPDATED DEPENDENCY INSTALL

```bash
# Add these to the existing npm install command:
npm install @supabase/supabase-js qrcode @types/qrcode

# Everything else is the same as the base document
```

UPDATED BUILD EXECUTION ORDER

Follow the base document phases exactly, then add:

```
PHASE 8 — ATLAS VOICE (after Phase 5)
→ /lib/atlas.ts
→ /lib/speech-synthesis.ts
→ /hooks/useAtlasVoice.ts
→ /components/ui/AtlasOrb.tsx
→ Wire into /app/universe/[documentId]/page.tsx

PHASE 9 — COLLABORATION (after Phase 8)
→ /lib/collaboration.ts
→ /hooks/useCollaboration.ts
→ /components/3d/CollaboratorCursor.tsx
→ /components/ui/CollabChat.tsx
→ Add studySessions + atlasHistory to schema, push migration
```

PHASE 10 — AR EXPORT (after Phase 9)
→ /app/ar/[documentId]/page.tsx
→ /components/3d/ARCanvas.tsx
→ Add AR button to universe top bar
→ Add QR code modal for desktop → mobile handoff

PHASE 11 — VISUAL POLISH (final, Day 2 morning)
→ NodeBurst particle effect
→ DynamicStars
→ Animated ConceptEdge beams
→ UniverseLoader screen
→ Test full demo flow: upload → universe → Atlas → collab → AR
```

---

## THE DEMO SCRIPT (memorise this)

**0:00 — Hook (15 seconds)**
"Every other project here is a quiz app. We built something different. Watch."

**0:15 — Upload moment**
Upload a lecture PDF on stage. Watch the processing animation. 3D universe appears.

**0:35 — Atlas moment**
Press the orb. Say: "Atlas, what should I study first?"
Atlas responds conversationally, the weakest node pulses red. Judges will gasp.

**0:55 — Collaboration moment**
Open the link on a second device. Two avatars appear in the same space.
"My friend can study in the same universe, in real time."

**1:10 — AR moment**
Scan QR with phone. Point at the table. Knowledge graph appears in the room.
Do not say anything. Let the visual do the work.

**1:25 — Memory system**
"Every node glows green, yellow, or red based on how well you remember it.
This is the first time anyone has visualised forgetting in 3D."

**1:40 — Close**
"This is Mnemo. Built in two days. Zero cost to run. Ready for real students tomorrow."

---
```

KNOWN ERRORS FOR NEW ADDITIONS

Error	Fix
Web Speech API not working	Only works on HTTPS. Vercel deploy solves this. Test locally with `npm run dev` using Chrome.
Speech synthesis no voice	Call `speechSynthesis.getVoices()` inside a user gesture (button click), not on mount — browsers block it otherwise
Supabase presence not syncing	Ensure channel name is identical across all clients: `universe:${documentId}` exactly
WebXR not starting	Must be on HTTPS and user must grant camera permission. Show clear permission prompt before starting AR
QR code not generating	Install `qrcode` and `@types/qrcode` — it's not in the base package list
Atlas returns non-JSON	Gemini occasionally wraps in markdown. The `.replace(/```json|```/g, "")` strip handles this — ensure it runs before `JSON.parse`
AR graph too large/small	Adjust `scale` prop in ARPage. Start at 0.15, tune to 0.1–0.2 for desk surface

---

JUDGE Q&A PREP

"How does the memory decay work?"
"We track every node visit, time spent, and quiz answer. A weighted formula combines accuracy and time-since-review to produce a 0–1 strength score. Nodes shift from green to red as memory fades. It's based on the forgetting curve model."

"Can this scale beyond a hackathon?"
"The entire stack costs zero to run — Gemini free tier handles 1,500 requests per day, Neon's free tier holds 512MB, Supabase handles 500 concurrent connections. To scale, we swap Gemini for a paid key and Neon for a larger instance. Architecture doesn't change."

"What makes this better than Anki or Notion AI?"
"Neither has a 3D spatial memory system. Spatial memory research shows 40% better retention when information is encoded to a location. Anki is a list. Notion is a document. This is a place you remember."

"Is the AR actually AR or just a camera background?"
"It uses WebXR hit-test, which detects real surfaces through the phone's depth sensors and places 3D objects on them with correct perspective and occlusion. It's the same API Apple and Google use for their AR experiences."

This document extends MNEMO-ANTIGRAVITY-MASTER. All base decisions stand. Only additions are here. Execute base phases first, then Phases 8–11.